International Institute of Information Technology Bangalore

CSE731: Software Testing – Mid-term Project, Term I (2026–27)

Agentic AI Pipeline for Coverage-Directed Unit Test Generation

Sanyam Verma IMT2023040
Gautam Kappagal IMT2023082

Testing requirement pursued: (1) user-specified coverage criterion — statement coverage and branch coverage, target 100%.

1 Pipeline, Dataset and Test-Generator Functionality

Dataset. MBPP (`google-research-datasets/mbpp`), `train` split, five problems (task IDs 601, 602, 604, 605×2). Task ID 603 ("lucid number") was skipped because all tested free models consistently returned syntactically invalid Python for that prompt; the pipeline's `-skip-mbpp-ids` flag substituted the next available problem. MBPP 605 was run twice independently (different random seeds / model assignments), providing a comparison of two generated implementations of the same specification. Only the natural-language task is given to the pipeline; MBPP's reference code and tests are stored alongside for traceability.

MBPP ID	Task
601	Write a function to find the longest chain which can be formed from the given set of pairs.
602	Write a python function to find the first repeated character in a given string.
604	Write a function to reverse words in a given string.
605 (trial A)	Write a function to check if the given integer is a prime number.
605 (trial B)	Write a function to check if the given integer is a prime number.

Table 1: MBPP problems used (five pipeline runs).

Agents. The pipeline is plain Python (one class per agent, a loop in `pipeline.py`, direct HTTPS calls to OpenRouter); no RAG, no agent framework and no chain-of-thought prompting is used.

1. **Code Generator Agent** (LLM): MBPP task → `solution.py`.
2. **Test Generator Agent** (LLM): task, code, and coverage criterion → `test_solution.py` (Python `unittest`).
3. **Test Executor Agent** (deterministic): runs the suite under `coverage.py` in branch mode and returns the verdict (PASS/FAIL/TIMEOUT), statement %, branch % and the uncovered lines/branches. Execution and coverage are measured by tools rather than an LLM so the verdict cannot be hallucinated.

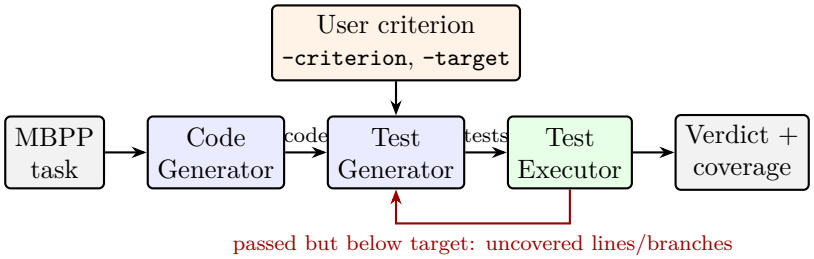


Figure 1: Pipeline. Blue: LLM agents; green: deterministic executor agent.

Test-generator functionality. The user selects the criterion (`statement+branch` (default), `statement` or `branch`) and a target. The prompt asks for representative, boundary and edge inputs, both outcomes of every conditional, and zero/one-or-more loop iterations. Coverage is computed as

$$\text{Stmt}\% = \frac{\text{statements} - \text{missing}}{\text{statements}} \times 100, \qquad \text{Branch}\% = \frac{\text{covered branches}}{\text{total branches}} \times 100.$$

If a suite passes but misses the target, the executor's list of uncovered statements and branch outcomes is sent back to the Test Generator to extend the suite (at most 2 rounds). Failing suites are not sent back, since a failure may reveal a real defect in the code. Generated output is rejected unless it parses (`ast.parse`), and test modules must use `unittest` and contain a `test_` method. On a validation failure the agent automatically retries with the next available free model, excluding the model that returned invalid output.

2 Prompts and Settings

Each agent fills a fixed template and sends it as a single `user` message (no system message, no history). The user input is the MBPP slice (`-n`) and the testing requirement (`-criterion`, `-target`).

```
You are the Code Generator Agent in an automated software-testing experiment.

Programming problem:
{task}

Generate a correct Python implementation for the problem.

Strict output rules:
- Return ONLY executable Python source code.
- Do not use Markdown code fences.
- Do not include explanations.
- Do not include JSON.
- Do not include safety classifications.
- Do not include analysis.
- Define the function(s) required by the problem.
- Keep the implementation concise and self-contained.
```

Listing 1: Code Generator prompt.

```
You are the Test Generator Agent in an AI-assisted unit-testing pipeline.

Programming problem:
{task}

Generated Python implementation:
{implementation}

Create a complete Python unittest module named test_solution.py.

Test-design requirements:
1. Use Python's built-in unittest framework.
2. Import the function(s) from solution.py.
3. Include normal/representative cases.
4. Include boundary cases where meaningful.
5. Include edge cases such as empty, singleton, negative,
   duplicate, or unusual inputs when applicable.
6. For each important conditional, exercise both outcomes.
7. For loops, exercise zero iterations and one-or-more iterations
   when meaningful.
8. Use strong assertions with expected outputs.
9. Do not test implementation details that are irrelevant to the
   problem specification.
```

```
10. {criterion_text}

Strict output rules:
- Return ONLY executable Python source code.
- Do not use Markdown code fences.
- Do not include explanations.
- Do not include JSON.
- Do not include safety classifications.
```

Listing 2: Test Generator prompt. Item 10 depends on the chosen criterion; for `statement+branch` it is "Aim for high statement and branch coverage."

The coverage-feedback prompt repeats the task, the code with line numbers and the current tests, adds "Statements never executed: ..." and "Branch outcomes never taken: ..." from the executor, and asks for the complete updated module.

Parameter	Value
LLM provider	OpenRouter free models (auto-discovered; excluded per-call on validation failure)
Models used	`nvidia/nemotron-3.5-lightning:free`, `qwen/qwen3.8-27b:free`, `nvidia/nemotron-3-ultra-550b-a55b:free`, `nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free`
Temperature	0.2 (all LLM calls)
`max_tokens`	1500 (code), 2500 (tests)
Other sampling parameters	provider defaults
HTTP timeout	10 s connect, 60 s read
Test framework / coverage	`unittest`; `coverage run -branch -source=solution`
Executor timeout	180 s per suite
Criterion / target	statement + branch, 100%

Table 2: Pipeline settings.

3 Format of Code, Tests and Verdict

Code: `generated/problem_N/solution.py`, a plain Python module with the required function (s). **Tests:** `generated/problem_N/test_solution.py`, a `unittest.TestCase` class with one `test_*` method per scenario using `assertEqual`/`assertTrue`/... with explicit expected values. **Verdict:** a JSON record saved in `experiment.json`:


```
{ "verdict": "PASS", "passed": true, "tests_run": 16,
  "statement_coverage": 100.0, "branch_coverage": 100.0,
  "missing_lines": [], "missing_branches": [],
  "execution_time_seconds": 0.1, "stderr": "Ran 16 tests in 0.001s  OK" }
```


Generated code and test cases

```python
def findLongestChain(pairs):
    pairs.sort(key=lambda x: x[1])
    count = 0
    curr = float('-inf')
    for a, b in pairs:
        if a > curr:
            count += 1
            curr = b
    return count
```

Listing 3: MBPP 601 — generated code

```python
import unittest
from solution import findLongestChain


class TestFindLongestChain(unittest.TestCase):
    def test_empty_list(self):
        self.assertEqual(findLongestChain([]), 0)

    def test_single_pair(self):
        self.assertEqual(findLongestChain([[1, 2]]), 1)

    def test_single_negative_pair(self):
        self.assertEqual(findLongestChain([[-5, -1]]), 1)

    def test_all_pairs_form_chain(self):
        self.assertEqual(findLongestChain([[1, 2], [3, 4], [5, 6]]), 3)

    def test_strict_greater_condition_true(self):
        self.assertEqual(findLongestChain([[1, 2], [3, 4]]), 2)

    def test_equal_start_condition_false(self):
        self.assertEqual(findLongestChain([[1, 2], [2, 3]]), 1)

    def test_duplicate_pairs(self):
        self.assertEqual(findLongestChain([[1, 2], [1, 2], [1, 2]]), 1)

    def test_unsorted_input(self):
        self.assertEqual(findLongestChain([[3, 4], [1, 2]]), 2)

    def test_negative_numbers_chain(self):
        self.assertEqual(findLongestChain([[-5, -3], [-2, 0], [1, 2]]), 3)

    def test_negative_numbers_no_chain(self):
        self.assertEqual(findLongestChain([[-5, -1], [-3, -2]]), 1)

    def test_mixed_overlapping_intervals(self):
        self.assertEqual(findLongestChain([[1, 5], [2, 3], [4, 6]]), 2)

    def test_longer_chain_skips_overlapping_pair(self):
        self.assertEqual(findLongestChain([[1, 10], [2, 3], [4, 5], [6, 7]]), 3)


if __name__ == '__main__':
    unittest.main()
```

Listing 4: MBPP 601 — generated tests

Missing: generated/problem_2/solution.py

Missing: generated/problem_2/test_solution.py

Missing: generated/problem_3/solution.py

Missing: generated/problem_3/test_solution.py

Missing: generated/problem_4/solution.py

Missing: generated/problem_4/test_solution.py

Missing: generated/problem_5/solution.py

Missing: generated/problem_5/test_solution.py

4 Execution Results

MBPP ID	Tests generated	Verdict	Statement cov.	Branch cov.	Runtime (s)
601	7	PASS	100%	100%	281.0
602	16	PASS	100%	100%	69.1
604	14	PASS	100%	100%	71.9
605 (A)	8	PASS	100%	100%	262.7
605 (B)	11	PASS	100%	100%	242.1
Total	**56**	**5/5**	**100%**	**100%**	**926.8**

Table 3: Results of the Test Generator and Test Executor agents. Runtime includes LLM latency.

All five suites met the criterion on the first attempt, so no coverage-feedback round was needed. Test execution itself took under a second per problem; nearly all runtime was LLM latency on free-tier models (e.g. MBPP 601: 56.5 s code generation, 224.0 s test generation, <0.1 s execution).

The two independent runs of MBPP 605 generated structurally different implementations (trial A uses the $6k \pm 1$ divisibility optimisation; trial B uses a simple odd-step loop) and different test suites (8 vs. 11 methods), both achieving 100% statement and branch coverage.

Limitations. 100% statement and branch coverage shows every statement and decision outcome was executed, not that the code is correct: the tests are written from the generated code, so a misunderstanding shared by both would still PASS. Task ID 603 ("lucid number") was skipped because free models consistently returned syntactically invalid Python for it, suggesting the task description is ambiguous or too obscure for small free-tier models. Running MBPP's reference tests or mutation testing would give an independent correctness signal.

5 Individual Contributions

Member	Contribution
Sanyam Verma (IMT2023040)	Selected the testing goal (coverage criterion) and dataset; built the Code Generator and Test Generator agents and the OpenRouter client; designed the test-generation prompt (boundary/edge cases, both outcomes of each decision, loop iterations) and the coverage-feedback loop; output validation of generated code and tests; per-call model exclusion on validation failure.
Gautam Kappagal (IMT2023082)	Built the Test Executor agent: running suites with `unittest` and `coverage.py`, statement/branch coverage computation, PASS/FAIL/TIMEOUT verdicts; pipeline orchestration and result logging; ran the experiment, analysed the coverage results and limitations, and wrote the report.